

Домашка 04 — Мережі. Частина 1

Автор: Андрій Пономарьов / Claude Code **Завдання:** пройти шлях `dig` → `route` → `traceroute` для довільного сайту, зняти `tcpdump` під час `curl` і знайти TCP handshake (SYN, SYN-ACK, ACK), записати у файл кількість хопів, шлюз і його MAC, пояснити, чому MAC змінюється, а IP — ні.

Обраний сайт: github.com

Примітка: робота виконувалась на macOS, де немає команди `ip route get` (вона з linux-пакета `iproute2`). Використано нативний еквівалент — `route -n get`.

1. dig — DNS-резолв

```
$ dig github.com

;; ANSWER SECTION:
github.com.      60 IN  A   140.82.121.4

;; Query time: 50 msec
;; SERVER: 192.168.50.2#53(192.168.50.2)
```

Що це означає: DNS-сервер 192.168.50.2 (локальний, у мережі роутера) відповів, що `github.com` зараз резолвиться в IP 140.82.121.4. TTL = 60 секунд — запис короткоживучий, GitHub ротує адреси між 140.82.121.3–.5 для балансування навантаження (в `traceroute` нижче DNS вже видав .3). Повний вивід: [docs/dig-output.txt](#).

2. route — куди піде пакет

```
$ route -n get github.com
route to: 140.82.121.4
gateway: 10.20.50.1
interface: en0
flags: <UP,GATEWAY,DONE,STATIC,PRCLONING,GLOBAL>
mtu: 1500
```

Що це означає: адреса 140.82.121.4 не в моїй локальній мережі, тому пакет піде через шлюз за замовчуванням 10.20.50.1 з інтерфейсу `en0` (Wi-Fi), MTU 1500 байтів. Повний вивід разом з `netstat -rn ta arp -a`: [docs/route-output.txt](#).

3. traceroute — шлях пакета

```
$ traceroute -I -n -w 2 -m 20 github.com
traceroute to github.com (140.82.121.3), 20 hops max, 48 byte packets
 1  10.20.50.1  2.872 ms  4.571 ms  4.676 ms
 2  10.12.18.1  4.468 ms  4.667 ms  4.415 ms
 3  10.1.1.1.3  4.604 ms  5.286 ms  5.603 ms
 4  91.226.254.3  5.074 ms  6.119 ms  3.371 ms
 5  185.151.86.16  4.652 ms  9.569 ms  6.392 ms
 6  185.1.226.180  56.857 ms  56.701 ms  55.604 ms
 7  80.81.196.80  44.399 ms  45.139 ms  46.686 ms
 8  * * *
 9  * * *
10  140.82.121.3  46.064 ms  48.998 ms  46.365 ms
```

Що це означає: до GitHub — **10 хопів**. Перший хоп — мій шлюз 10.20.50.1, хопи 2–3 — приватні адреси мережі провайдера, з хопа 4 починаються публічні адреси. Стрибок затримки з ~5 мс до ~55 мс на хопі 6 (185.1.226.180 — це peering-точка DETEL-IX) — момент, коли трафік виходить за кордон. Хопи 8–9 (* * *) — маршрутизатори GitHub, які не відповідають на ICMP TTL exceeded. Звичайний `traceroute -n` (UDP-проби) показує лише перші 7 хопів — GitHub відкидає UDP на високі порти, тому фінал шляху видно тільки по ICMP (-I). Повний вивід обох варіантів: [docs/traceroute-output.txt](#).

4. tcpdump під час curl — TCP handshake

Захоплення на `en0` з фільтром на мережу GitHub (140.82.112.0/20), паралельно виконувався `curl -s -o /dev/null https://github.com`:

```
$ sudo tcpdump -i en0 -nn "tcp and net 140.82.112.0/20"

19:32:46.921664 IP 10.20.50.89.52079 > 140.82.121.3.443: Flags [SEW], seq 1977315521, win 6553
5, options [mss 1460,...], length 0
19:32:46.967969 IP 140.82.121.3.443 > 10.20.50.89.52079: Flags [S.E], seq 67592, ack 197731552
```

```
2, win 65535, options [mss 1436,...], length 0
19:32:46.968157 IP 10.20.50.89.52079 > 140.82.121.3.443: Flags [.], ack 1, win 2070, options
[...], length 0
```

Що це означає: це і є three-way handshake:

1. **SYN** — `Flags [SEW]`: мій комп'ютер (10.20.50.89, порт 52079) шле SYN на 140.82.121.3:443 з початковим seq 1977315521. Літера s — SYN, E і W (ECE+CWR) — узгодження ECN, розширення для контролю перевантаження.
2. **SYN-ACK** — `Flags [S.E]`: сервер відповідає своїм SYN (seq 67592) і підтверджує мій — ack 1977315522 (мій seq + 1). Крапка у прапорах tcpdump означає ACK.
3. **ACK** — `Flags [.]`: я підтверджую SYN сервера, з'єднання встановлене. Handshake зайняв ~46 мс — рівно один RTT до GitHub (збирається з traceroute).

Одразу після цього йде пакет `Flags [P.]`, `length 320` — це вже TLS ClientHello поверх встановленого з'єднання. Ключові фрагменти дампа: [docs/tcpdump-output.txt](#).

5. Файл з підсумком

Кількість хопів, шлюз і його MAC записані у [docs/network-info.txt](#):

- **Хопів до github.com**: 10
- **Шлюз**: 10.20.50.1 (en0)
- **MAC шлюзу**: b0:e4:d5:43:f3:c8

6. Чому MAC змінюється, а IP — ні

IP-адреси в заголовку пакета — це адреси кінцевих точок з'єднання (мій комп'ютер і сервер GitHub), вони працюють на мережевому рівні (L3) і задають маршрут «звідки й куди» на всьому шляху. MAC-адреси працюють на канальному рівні (L2) і мають сенс лише в межах одного локального сегмента мережі — наприклад, між моїм ноутбуком і Wi-Fi-роутером. Кожен маршрутизатор на шляху розпаковує Ethernet-кадр, дивиться на IP-адресу призначення і запаковує пакет у новий кадр: source MAC стає адресою самого маршрутизатора, а destination MAC — адресою наступного хопу (яку він дізнається через ARP). Тому на кожному з 10 хопів пара MAC-адрес інша, а пара IP-адрес у заголовку пакета лишається незмінною від початку до кінця. Єдиний виняток — NAT на домашньому роутері, який підмінює мій приватний source IP (10.20.50.x) на публічну адресу, але destination IP сервера все одно не змінюється ніде.

Опціональна частина

7. nc -zv: відкритий / закритий / фільтрований порт

```
$ nc -zv github.com 443
Connection to github.com port 443 [tcp/https] succeeded!

$ nc -zv 127.0.0.1 65000
nc: connectx to 127.0.0.1 port 65000 (tcp) failed: Connection refused

$ time nc -zv -G 5 github.com 8080
nc: connectx to github.com port 8080 (tcp) failed: Operation timed out
(5.013 total)
```

Порівняння: три стани порту дають три різні реакції:

Порт	Результат	Час	Що сталося на рівні TCP
github.com:443 (відкритий)	succeeded	миттєво	SYN → SYN-ACK
127.0.0.1:65000 (закритий)	Connection refused	миттєво	SYN → RST
github.com:8080 (фільтрований)	Operation timed out	5 с (таймаут)	SYN → тиша

Закритий порт «чесний» — ОС одразу відповідає RST. Фільтрований — firewall у режимі DROP мовчки з'їдає SYN, і клієнт здається лише по таймауту. Саме за цією різницею nmap відрізняє closed від filtered. Повний вивід: [docs/nc-output.txt](#).

8. MTU і ping -s

MTU en0 = 1500 (видно у `route -n get`). Розмір ICMP-пакета = payload + 8 (ICMP) + 20 (IP), тож межа payload без фрагментації: $1500 - 28 = 1472$. Прапор `-D` ставить DF-біт (Don't Fragment):

```
$ ping -c 3 -D -s 1472 github.com      # 1472+28 = 1500 - рівно MTU
1480 bytes from 140.82.121.4: ... time=46.133 ms  -> 0% loss

$ ping -c 3 -D -s 1473 github.com      # 1501 > MTU, DF заборонив різати
ping: sendto: Message too long          -> 100% loss

$ ping -c 3 -s 2000 github.com          # 2028 > MTU, але БЕЗ DF
2008 bytes from 140.82.121.4: ... time=45.309 ms  -> 0% loss
```

Що це означає: 1472 байти проходять рівно в MTU; +1 байт з DF-бітом — ядро навіть не відправляє пакет («Message too long»); той самий великий пакет без DF проходить, бо IP-рівень прозора розрізає його на два фрагменти, а приймальна сторона склеює (RTT не змінився). На цьому механізмі побудовано Path MTU Discovery: TCP завжди ставить DF і зменшує сегмент, коли з маршруту прилітає ICMP “fragmentation needed”. Повний вивід: [docs/mtu-ping-output.txt](https://docs.mtu-ping-output.txt).

9. Wireshark — той самий handshake у GUI

Встановлено Wireshark 4.6.7 (`brew install --cask wireshark`; разом з ним ставиться ChmodBPF, після чого захоплення працює без `sudo`). Запуск одразу із захопленням і тим самим фільтром, що в `tcpdump`:

```
$ open -a Wireshark --args -i en0 -k -f "tcp and net 140.82.112.0/20"
$ curl -s -o /dev/null https://github.com
```

Перші три пакети списку — той самий three-way handshake, четвертий — TLS Client Hello з `SNI=github.com`:

Apply a display filter ...				
No.	Time	Source	Destination	
1	0.000000	10.20.50.89	140.82.121.4	
2	0.056079	140.82.121.4	10.20.50.89	
3	0.056222	10.20.50.89	140.82.121.4	
4	0.056368	10.20.50.89	140.82.121.4	
5	0.116946	140.82.121.4	10.20.50.89	
6	0.116950	140.82.121.4	10.20.50.89	
7	0.116952	140.82.121.4	10.20.50.89	
8	0.117163	10.20.50.89	140.82.121.4	
9	0.117185	10.20.50.89	140.82.121.4	
10	0.119482	10.20.50.89	140.82.121.4	
11	0.119498	10.20.50.89	140.82.121.4	
12	0.119550	10.20.50.89	140.82.121.4	
13	0.119584	10.20.50.89	140.82.121.4	
14	0.176914	140.82.121.4	10.20.50.89	
15	0.176916	140.82.121.4	10.20.50.89	
16	0.176918	140.82.121.4	10.20.50.89	
17	0.176919	140.82.121.4	10.20.50.89	
18	0.176921	140.82.121.4	10.20.50.89	
19	0.177092	10.20.50.89	140.82.121.4	
20	0.177136	10.20.50.89	140.82.121.4	
21	0.177303	10.20.50.89	140.82.121.4	
22	0.182653	140.82.121.4	10.20.50.89	
23	0.182656	140.82.121.4	10.20.50.89	
24	0.182658	140.82.121.4	10.20.50.89	
25	0.182659	140.82.121.4	10.20.50.89	
26	0.182661	140.82.121.4	10.20.50.89	
27	0.182662	140.82.121.4	10.20.50.89	
> Frame 1: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface 0 > Ethernet II, Src: 46:c0:a4:4b:9b:6b (46:c0:a4:4b:9b:6b), Dst: b0:e4:d5:43:f3:c8 (b0:e4:d5:43:f3:c8) > Internet Protocol Version 4, Src: 10.20.50.89, Dst: 140.82.121.4 > Transmission Control Protocol, Src Port: 54456, Dst Port: 80				

Handshake у Wireshark

Що видно: Wireshark розписує прапори повними назвами — те, що tcpdump показував як [SEW] / [S.E] / [.] , тут підписано [SYN, ECE, CWR] / [SYN, ACK, ECE] / [ACK]. У панелі деталей для Frame 1 (SYN) видно Ethernet-заголовок: Src 46:c0:a4:4b:9b:6b (MAC ноутбука) → Dst b0:e4:d5:43:f3:c8 —

це MAC шлюзу з розділу 5, жива ілюстрація до пояснення «MAC vs IP»: кадр до GitHub фізично адресований роутеру. Wireshark навіть підказує вендора шлюзу — Google_43:f3:c8 (Nest Wifi).

Дзеркальна картина у Frame 2 (SYN-ACK): там source MAC — вже шлюз b0:e4:d5:43:f3:c8, а destination — ноутбук ([docs/wireshark-synack-frame.png](#), [docs/wireshark-ack-frame.png](#)). Кінець тієї ж сесії — обмін [FIN, ACK] з обох боків і фінальний [RST] (закриття з'єднання): [docs/wireshark-session.png](#).

Звіт (підсумок)

Пункт	Результат
Сайт	github.com → 140.82.121.3/4
Хопів	10 (ICMP traceroute)
Шлюз	10.20.50.1 (en0)
MAC шлюзу	b0:e4:d5:43:f3:c8
Handshake	SYN → SYN-ACK → ACK (tcpdump, розділ 4; Wireshark, розділ 9)